

Experiment 'p_6_2_f_move_even_to_front_seq' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_2_f_move_even_to_front_seq

Natural language statement: Write a method to move all even elements in an array to the front, otherwise preserving the order of the elements.

Method signature: p_6_2_f_move_even_to_front_seq(inputs: seq<int>) returns (result: seq<int>)

Ensures

- result == evens(inputs) + odds(inputs)

Functional Code Given

```
function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
    then [s[0]] + evens(s[1..])
    else evens(s[1..])
}

function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
    then [s[0]] + odds(s[1..])
    else odds(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to move all even elements in an array to the front, otherwise preserving
→ the order of the elements.

The signature should be:

`p_6_2_f_move_even_to_front_seq(inputs: seq<int>) returns (result: seq<int>)`

The method should respect the following contract:

`ensures result == evens(inputs) + odds(inputs)`

The contract uses the following dafny code:

```
function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
    then [s[0]] + evens(s[1..])
    else evens(s[1..])
}

function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
    then [s[0]] + odds(s[1..])
    else odds(s[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn0wO1bA1G3JVzIg5PVbCojGfpYsk

Model: gpt-5.5-2026-04-23

Created at: 1780574060

Finish reason: stop

Usage: 580 tokens in, and 5488 tokens out

```

{
  var es: seq<int> := [];
  var os: seq<int> := [];
  var i := |inputs|;

  while i > 0
    invariant 0 <= i <= |inputs|
    invariant es == evens(inputs[i..])
    invariant os == odds(inputs[i..])
  {
    i := i - 1;

    assert 0 <= i && i < |inputs|;
    assert inputs[i..i+1] == [inputs[i]];
    assert inputs[i..] == inputs[i..i+1] + inputs[i+1..];
    assert inputs[i..] == [inputs[i]] + inputs[i+1..];
    assert (inputs[i..])[0] == inputs[i];
    assert (inputs[i..])[1..] == inputs[i+1..];

    if inputs[i] % 2 == 0 {
      assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i+1..]);
      assert odds(inputs[i..]) == odds(inputs[i+1..]);
      es := [inputs[i]] + es;
    } else {
      assert evens(inputs[i..]) == evens(inputs[i+1..]);
      assert odds(inputs[i..]) == [inputs[i]] + odds(inputs[i+1..]);
      os := [inputs[i]] + os;
    }

    assert es == evens(inputs[i..]);
    assert os == odds(inputs[i..]);
  }

  assert i == 0;
  assert inputs[i..] == inputs;
  assert es == evens(inputs);
  assert os == odds(inputs);
  result := es + os;
}

```

Final Program

```

function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
    then [s[0]] + evens(s[1..])
    else evens(s[1..])
}

```

```

function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
    then [s[0]] + odds(s[1..])
    else odds(s[1..])
}

method p_6_2_f_move_even_to_front_seq(inputs: seq<int>) returns (result: seq<int>)
  ensures result == evens(inputs) + odds(inputs)
{
  var es: seq<int> := [];
  var os: seq<int> := [];
  var i := |inputs|;

  while i > 0
    invariant 0 <= i <= |inputs|
    invariant es == evens(inputs[i..])
    invariant os == odds(inputs[i..])
  {
    i := i - 1;

    assert 0 <= i && i < |inputs|;
    assert inputs[i..i+1] == [inputs[i]];
    assert inputs[i..] == inputs[i..i+1] + inputs[i+1..];
    assert inputs[i..] == [inputs[i]] + inputs[i+1..];
    assert (inputs[i..])[0] == inputs[i];
    assert (inputs[i..])[1..] == inputs[i+1..];

    if inputs[i] % 2 == 0 {
      assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i+1..]);
      assert odds(inputs[i..]) == odds(inputs[i+1..]);
      es := [inputs[i]] + es;
    } else {
      assert evens(inputs[i..]) == evens(inputs[i+1..]);
      assert odds(inputs[i..]) == [inputs[i]] + odds(inputs[i+1..]);
      os := [inputs[i]] + os;
    }

    assert es == evens(inputs[i..]);
    assert os == odds(inputs[i..]);
  }

  assert i == 0;
  assert inputs[i..] == inputs;
  assert es == evens(inputs);
  assert os == odds(inputs);
  result := es + os;
}

```

Total Token Usage

Input tokens: 580

Output tokens: 5488

Reasoning tokens: 5023

Sum of 'total tokens': 6068

Experiment Timings

Overall Experiment started at 1780574058913, ended at 1780574223650, lasting 164737ms (164.74 seconds)

Iteration #1 started at 1780574058916, ended at 1780574223649, lasting 164733ms (164.73 seconds)